

Reinforcement Learning Assignment Report

1. Introduction

This report presents the implementation of a reinforcement learning agent to solve a 5x5 maze navigation task via a remote API-based Gym environment. The goal was to train agents using two popular RL algorithms-DQN and PPO-such that they achieve at least 80 mean reward and reach the goal in no more than 20 steps on average.

2. Environment Setup

The environment was accessed via a custom Gymnasium wrapper class interfacing with the API. The following libraries were used: Gymnasium, Stable-Baselines3, Requests. All API calls were validated and functioned correctly during training and evaluation.

3. Algorithms Implemented

Deep Q-Network (DQN)

Deep Q-Network (DQN) is a reinforcement learning algorithm that combines Q-learning with deep neural networks. In traditional Q-learning, a value function (Q-function) estimates the expected rewards for state-action pairs. DQN enhances this by using a deep neural network to approximate this Q-function, allowing it to handle high-dimensional observation spaces such as images or complex sensor data. It uses experience replay and fixed target networks to stabilize training, reducing issues such as oscillation and divergence commonly seen in naive implementations.

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy-gradient method used in reinforcement learning, known for its stability and performance. PPO optimizes a parameterized policy directly by performing updates within a clipped objective function, ensuring changes to the policy remain controlled and stable. It avoids large updates to the policy that could negatively impact performance by maintaining a trust region for updates, which makes it easier to train compared to other policy gradient methods.

PPO has become popular due to its simplicity, strong performance on various tasks, and its robustness to hyperparameter choices.

4. Training Procedure

Agents were trained for 10,000 steps with continuous monitoring via TensorBoard. The mean episode reward was tracked for each algorithm. Hyperparameters were tuned iteratively to maximize performance and stability.

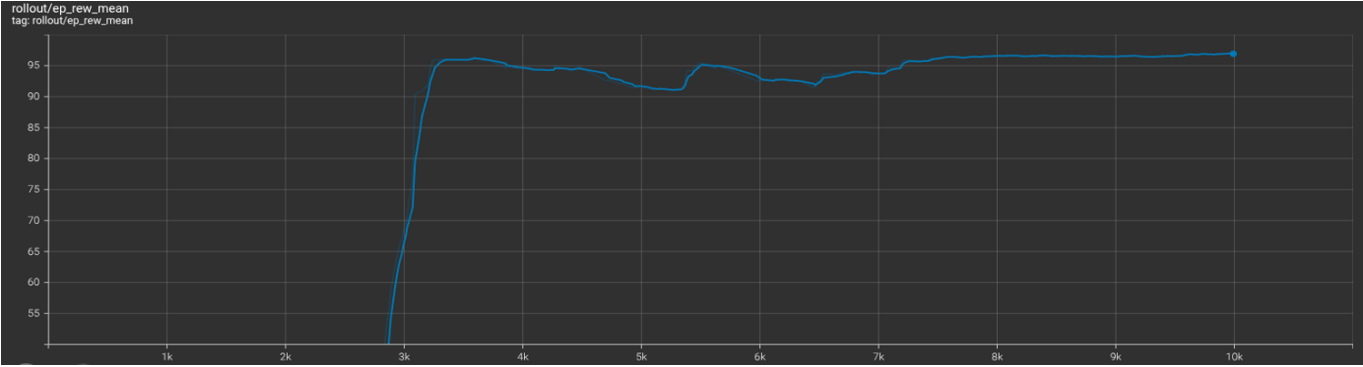
5. Results

Both algorithms successfully reached the required performance criterion (mean reward above 80). PPO (red) converged rapidly within the first 2,000 steps, achieving a stable reward of approximately 95-97. DQN (blue) showed slightly slower initial convergence, stabilizing around 95-96 rewards after 3,000 steps. Overall, PPO demonstrated marginally higher mean reward (96.68 - 1.70) compared to DQN (96.45 - 1.93), indicating greater consistency.

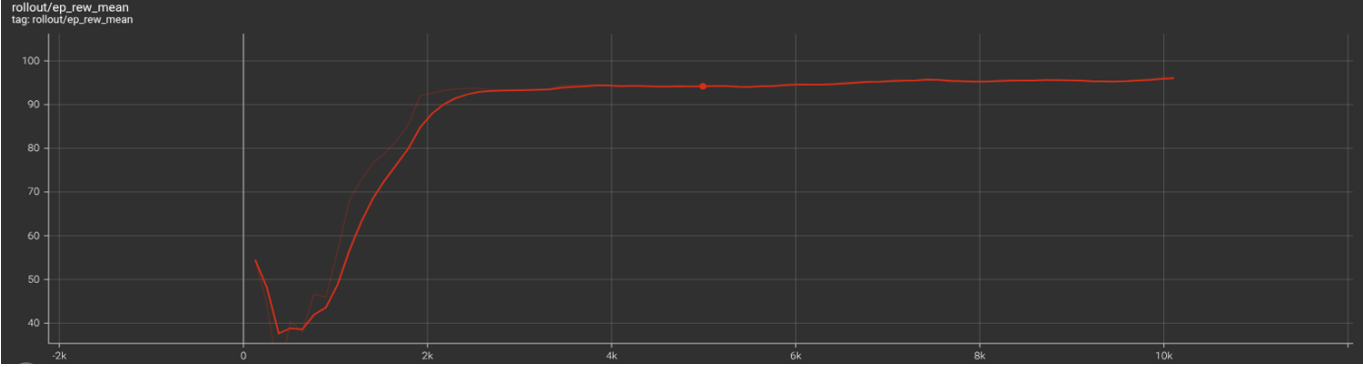
The mean episode length plots indicate both agents effectively learned to navigate the maze quickly. PPO (red) rapidly decreased its mean episode length within the initial 2,000 steps, stabilizing near an average of 5 steps per episode. DQN (blue) exhibited a quicker initial reduction, reaching an average of approximately 5 steps slightly earlier than PPO, maintaining consistent performance afterward.

The FPS plots indicate that PPO (red) maintained relatively stable performance, beginning around 240-250 FPS and gradually declining to around 230 FPS by the end of training. DQN (blue) initially exhibited higher performance (~280 FPS) but showed a consistent decrease throughout training, settling close to PPO's final FPS value around 220. The decline in FPS over time for both algorithms likely reflects increased computational overhead as policies become more complex and the computational graph grows.

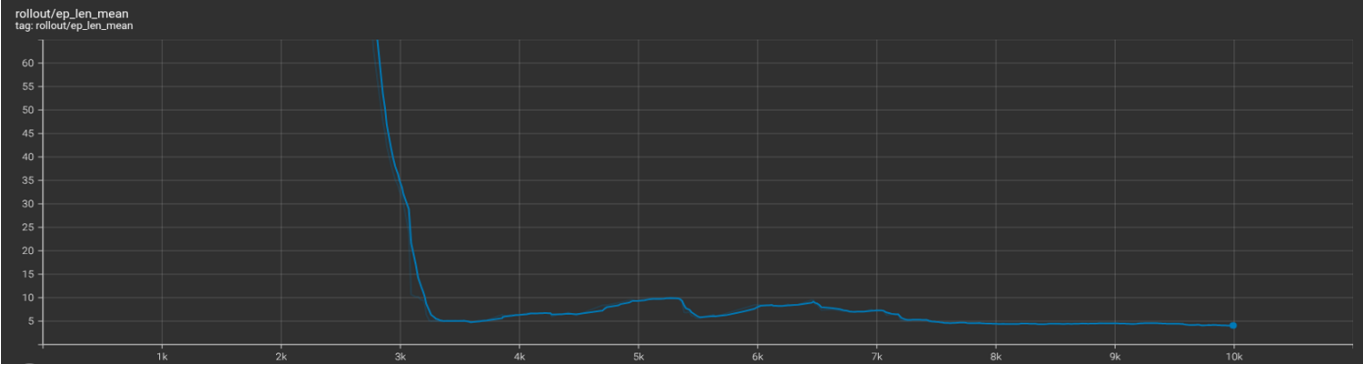
DQN (blue) - Mean Episode Reward



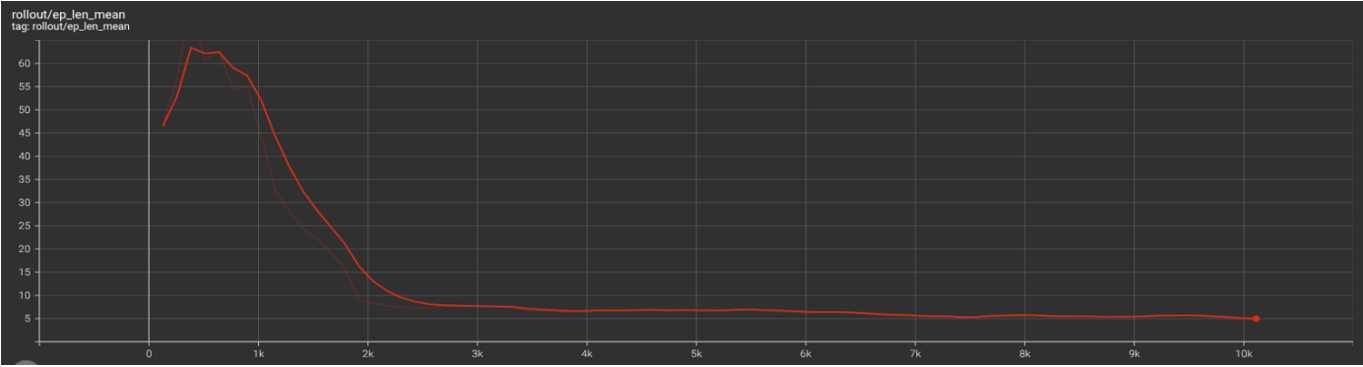
PPO (red) - Mean Episode Reward



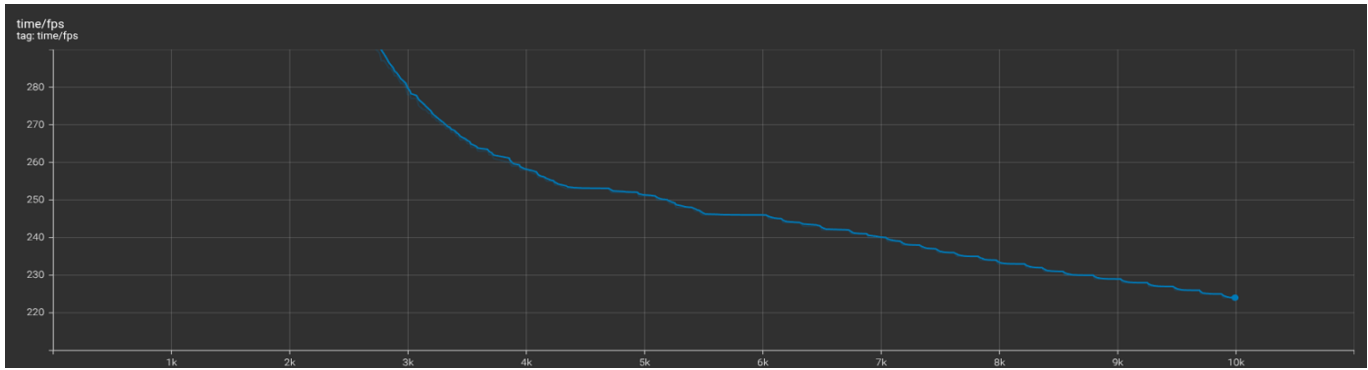
DQN (blue) - Mean Episode Length



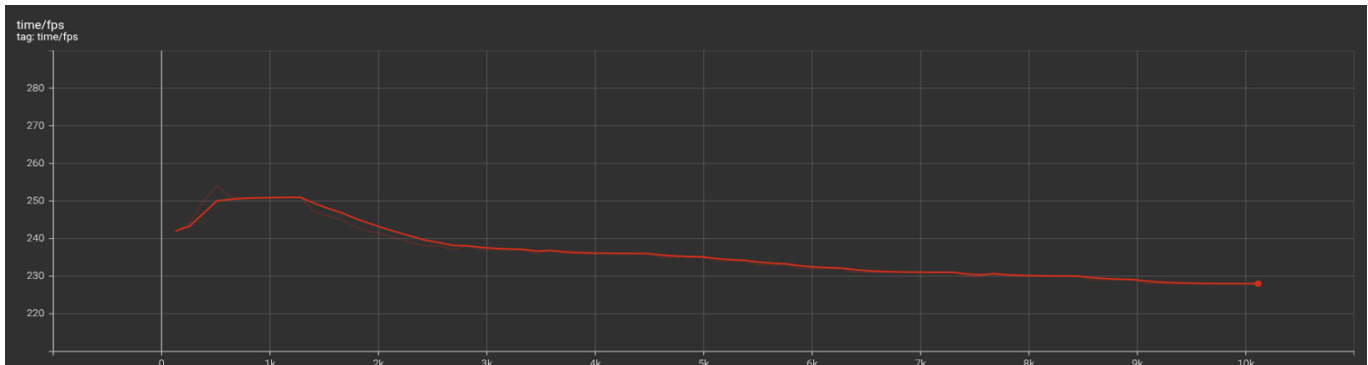
PPO (red) - Mean Episode Length



DQN (blue) - Training Speed (Frames per Second)



PPO (red) - Training Speed (Frames per Second)



6. Overall Analysis

Both PPO and DQN effectively solved the maze navigation task. PPO exhibited slightly superior reward performance and stability, while DQN showed faster initial performance improvement in terms of episode length. FPS measurements suggest similar computational demands for both algorithms, with a slight edge to PPO in sustained performance.

7. How to Run

To build the environment and dependencies, use the following command:

```
make build
```

To train the DQN agent, use:

```
make train-dqn
```

To train the PPO agent, use:

```
make train-ppo
```

To evaluate both trained agents, execute:

```
make evaluate
```

To view the training metrics via TensorBoard, open the following URL in your web browser:

```
http://localhost:6006/
```

To clean up generated files and reset the environment, use:

```
make clean
```